

Physical Simulation of a Couette Viscometer

Coding Weeks Project — CentraleSupélec / Université Paris-Saclay

November 2025

Even Baudet Paul Le Borgne Geoffroy de La Faire
Tom Aunis Ryad Badis Yanis Boucherk-Barthomeuf

Abstract

This report presents the design and implementation of an interactive physical simulation of a Couette viscometer — a device used to measure the viscosity of fluids by confining them between two coaxial rotating cylinders. The project was developed during a one-week intensive coding sprint at CentraleSupélec. The core challenge was to numerically solve the governing partial differential equation (PDE) of the tangential velocity field in a stable and efficient manner. We chose an **implicit backward Euler scheme**, which produces an unconditionally stable tridiagonal linear system solved at each time step via sparse matrix methods. The simulation features both Eulerian and Lagrangian visualisations of the flow, interactive parameter controls, and real-time plots of the angular velocities and velocity profile.

0 Contents

1	Introduction	2
2	Physical Background	2
2.1	Geometry and assumptions	2
2.2	Governing equation	2
2.3	Boundary and initial conditions	2
3	Numerical Method: Implicit Backward Euler	3
3.1	Why implicit?	3
3.2	Spatial discretisation	3
3.3	The tridiagonal system	3
3.4	Matrix assembly	4
3.5	Time-stepping loop	4
3.6	Stability and accuracy	5
4	Software Architecture	5
4.1	Overview	5
4.2	Object design	5
4.3	Visualisation modes	5
4.4	User interface	6
5	Results	7
5.1	Velocity profile convergence	7
5.2	Angular velocity of the outer cylinder	7
5.3	Numerical stability	7
6	Conclusion	7
A	Key Equations Summary	8

1 Introduction

A **Couette viscometer** consists of two coaxial cylinders of radii $R_1 < R_2$, with a viscous fluid filling the annular gap between them. The inner cylinder rotates at angular velocity $\omega_1(t)$, which can be prescribed as a constant, sinusoidal, or square-wave signal. The outer cylinder, initially at rest, begins to rotate due to viscous drag — the rate at which angular momentum is transmitted through the fluid is directly related to the kinematic viscosity ν .

This geometry is a classical benchmark in fluid mechanics: it admits an exact steady-state analytical solution, making it an excellent test case for numerical methods. Our project turns this setup into an **interactive Python application** built with Matplotlib widgets, combining real-time numerical solving with both Eulerian (velocity field) and Lagrangian (particle tracking) visualisations.

The rest of this report is structured as follows. Section 2 derives the governing PDE. Section 3 is the main focus: it details the discretisation strategy and the construction of the implicit linear system. Section 4 describes the software architecture. Section 5 presents the key simulation outputs. Section 6 concludes.

2 Physical Background

2.1 Geometry and assumptions

We work in cylindrical coordinates (r, θ, z) . The fluid is assumed incompressible and Newtonian. By symmetry, the velocity field has only a tangential component $v_\theta(r, t)$, with no dependence on θ or z . The radial and axial velocity components are zero.

2.2 Governing equation

Substituting the above assumptions into the Navier–Stokes equations reduces the problem to a single PDE for v_θ :

$$\frac{\partial v_\theta}{\partial t} = \nu \left(\frac{\partial^2 v_\theta}{\partial r^2} + \frac{1}{r} \frac{\partial v_\theta}{\partial r} - \frac{v_\theta}{r^2} \right), \quad (1)$$

where ν is the kinematic viscosity of the fluid. This is a diffusion-type PDE in the radial direction with a centrifugal correction term $-v_\theta/r^2$.

2.3 Boundary and initial conditions

- **Inner boundary** ($r = R_1$): no-slip condition, $v_\theta(R_1, t) = \omega_1(t) R_1$.
- **Outer boundary** ($r = R_2$): the outer cylinder is initially free to rotate. We impose a zero-gradient (Neumann) condition $\partial v_\theta / \partial r|_{R_2} \approx 0$, approximated as $v_{N_r-1}^n = v_{N_r-2}^n$ in the discrete setting. This models a freely dragged outer wall.
- **Initial condition**: $v_\theta(r, 0) = 0$ everywhere (fluid at rest).

The angular velocity of the outer cylinder $\omega_2(t)$ is recovered at each time step from the boundary value: $\omega_2(t) = v_\theta(R_2, t)/R_2$.

3 Numerical Method: Implicit Backward Euler

3.1 Why implicit?

An **explicit** (forward Euler) scheme evaluates the right-hand side at the current time step n :

$$v_i^{n+1} = v_i^n + \Delta t \cdot F(v^n),$$

where F encodes the spatial derivatives. This is straightforward to implement but imposes a severe stability constraint on the time step:

$$\Delta t \lesssim \frac{(\Delta r)^2}{2\nu},$$

which becomes prohibitively small for fine radial grids or large viscosities. As shown in an early prototype, this led to violently oscillating velocity profiles (see Figure ??).

An **implicit** (backward Euler) scheme instead evaluates F at the *unknown* time step $n + 1$:

$$v_i^{n+1} = v_i^n + \Delta t \cdot F(v^{n+1}).$$

This trades a trivial update for a linear system solve, but gains **unconditional stability** — the scheme remains well-behaved for any Δt and any Δr .

3.2 Spatial discretisation

The radial domain $[R_1, R_2]$ is discretised into N_r uniformly spaced points with step $\Delta r = (R_2 - R_1)/(N_r - 1)$. The grid points are $r_i = R_1 + i \Delta r$ for $i = 0, 1, \dots, N_r - 1$.

The three spatial derivative terms in Eq. (1) are discretised using second-order finite differences at time level $n + 1$:

$$\left. \frac{\partial^2 v_\theta}{\partial r^2} \right|_i \approx \frac{v_{i+1}^{n+1} - 2v_i^{n+1} + v_{i-1}^{n+1}}{(\Delta r)^2}, \quad (2)$$

$$\left. \frac{1}{r_i} \frac{\partial v_\theta}{\partial r} \right|_i \approx \frac{1}{r_i} \cdot \frac{v_{i+1}^{n+1} - v_{i-1}^{n+1}}{2 \Delta r}, \quad (3)$$

$$\left. \frac{v_\theta}{r_i^2} \right|_i \approx \frac{v_i^{n+1}}{r_i^2}. \quad (4)$$

3.3 The tridiagonal system

Substituting into the time-discrete equation and grouping unknowns at level $n + 1$, we define the dimensionless ratio

$$r_{\text{diff}} = \frac{\nu \Delta t}{(\Delta r)^2},$$

and obtain for each interior point $i \in \{1, \dots, N_r - 2\}$:

$$\left[1 + 2r_{\text{diff}} + \frac{\nu \Delta t}{r_i^2} \right] v_i^{n+1} - r_{\text{diff}} \left[1 + \frac{\Delta r}{2r_i} \right] v_{i+1}^{n+1} - r_{\text{diff}} \left[1 - \frac{\Delta r}{2r_i} \right] v_{i-1}^{n+1} = v_i^n. \quad (5)$$

This couples each unknown only to its two immediate neighbours, producing a **tridiagonal linear system**:

$$M \mathbf{v}^{n+1} = \mathbf{b}^n, \quad (6)$$

where $M \in \mathbb{R}^{N_r \times N_r}$ is assembled from the stencil coefficients, and $\mathbf{b}^n = \mathbf{v}^n$ with boundary corrections applied (the inner boundary value $\omega_1(t) R_1$ overwrites the first component, and the Neumann condition is encoded in the last row of M).

3.4 Matrix assembly

The matrix M is initialised as the identity (accounting for the v_i^n term on the right-hand side), then the interior rows are filled with the stencil of Eq. (5):

```

1 from scipy.sparse import identity
2 from scipy.sparse.linalg import spsolve
3
4 # M initialised as identity (right-hand side = v^n by default)
5 M = identity(nr, format='csc')
6
7 for i in range(1, nr - 1):
8     r_i = r_grid_1d[i]
9
10    # Sub-diagonal (i, i-1)
11    M[i, i-1] = -r_diff * (1 - dr / (2 * r_i))
12
13    # Main diagonal (i, i)
14    M[i, i] = 1 + 2 * r_diff + nu * dt / r_i**2
15
16    # Super-diagonal (i, i+1)
17    M[i, i+1] = -r_diff * (1 + dr / (2 * r_i))

```

Listing 1: Sparse matrix assembly (Python/SciPy)

The boundary rows are handled separately:

- **Row 0** (inner wall): $M[0, :] = \delta_{0,j}$, $b[0] = \omega_1(t) R_1$ — Dirichlet condition.
- **Last row** (outer wall): encodes $v_{N_r-1}^{n+1} - v_{N_r-2}^{n+1} = 0$ — zero-gradient Neumann condition.

3.5 Time-stepping loop

At each animation frame the system is solved via SciPy's sparse direct solver, which exploits the tridiagonal sparsity for $O(N_r)$ complexity:

```

1 def update_u(u):
2     b = u.copy()
3     b[0] = omega_1 * R1          # inner boundary condition
4     u = spsolve(M, b)          # solve M v^{n+1} = b^n
5     u[nr-1] = u[nr-2]          # enforce Neumann at outer wall
6     return u

```

Listing 2: One time step update

Because M does not change between time steps (for fixed parameters), it is factorised *once* and reused, making each update extremely fast.

3.6 Stability and accuracy

The backward Euler scheme is **first-order accurate in time** and **second-order accurate in space**. Its key advantage is unconditional stability: no matter how large Δt is chosen, the solution remains bounded. This is essential for an interactive simulation where the user may choose coarse time steps to speed up the animation. In practice, with $\Delta t \approx 21$ ms and $N_r = 30$ points, the solver runs well within real-time constraints.

4 Software Architecture

4.1 Overview

The project is written in Python and structured around three main modules, separating concerns between physics, numerics, and display:

- `obj.py` — defines the `Viscosimetre` class, which stores the full state of the system (radii, viscosity, velocity array, angular velocity histories, etc.) and exposes methods to update and query it.
- `eulerback.py` — defines the `BackwardsEuler` class, which owns the sparse matrix M , rebuilds it on parameter changes, and implements `calc_next()` to advance the solution by one time step.
- `main.py` — handles the entire Matplotlib figure: the Couette cross-section animation, the three live plots ($v(r)$, $\omega_1(t)$, $\omega_2(t)$), and all interactive widgets (sliders, radio buttons, text box, play/pause button).

This separation means the solver can be swapped out (e.g., replacing backward Euler with Crank–Nicolson) without touching the display code.

4.2 Object design

Class	Key attributes	Key methods
<code>Viscosimetre</code>	$\nu, R_1, R_2, N_r, \mathbf{v}, \omega_1\text{-hist}, \omega_2\text{-hist}$	<code>rebuild()</code> , <code>update()</code> , <code>current_omega2()</code>
<code>BackwardsEuler</code>	sparse matrix M	<code>build_matrice()</code> , <code>calc_next()</code>

The `Viscosimetre` delegates numerical work to its `resolv` attribute (an instance of `BackwardsEuler`), calling `self.v = self.resolv.calc_next()` at each time step. When a parameter slider is changed, `rebuild()` is called on the viscometer, which triggers `reload()` on the solver so that M is recomputed before the next step.

4.3 Visualisation modes

Two complementary views of the flow are provided:

- **Eulerian view:** velocity vectors are drawn on a fixed grid of points in the annular region, with direction and magnitude updated at each frame. This shows the instantaneous structure of the velocity field.
- **Lagrangian view:** fluid particles are initialised at fixed positions and advected using the local velocity $v_\theta(r_i, t)$. Their trajectories reveal the history of the flow and make the differential rotation between inner and outer regions intuitive.

4.4 User interface

Figure 1 shows the final interface. The left panel displays the viscometer cross-section with animated particles or velocity vectors. The right panel shows three live plots. The bottom panel exposes the following controls:

- **Signal type** (radio): sinusoidal or square-wave drive for $\omega_1(t)$.
- **Frequency** and **Amplitude** sliders: control the inner cylinder's rotation.
- R_1, R_2 **sliders**: inner and outer cylinder radii.
- ν **text box**: kinematic viscosity in units of $10^{-5} \text{ m}^2 \text{ s}^{-1}$.
- N_r **slider**: number of radial discretisation points.
- **Play/Pause button**: controls the animation loop.

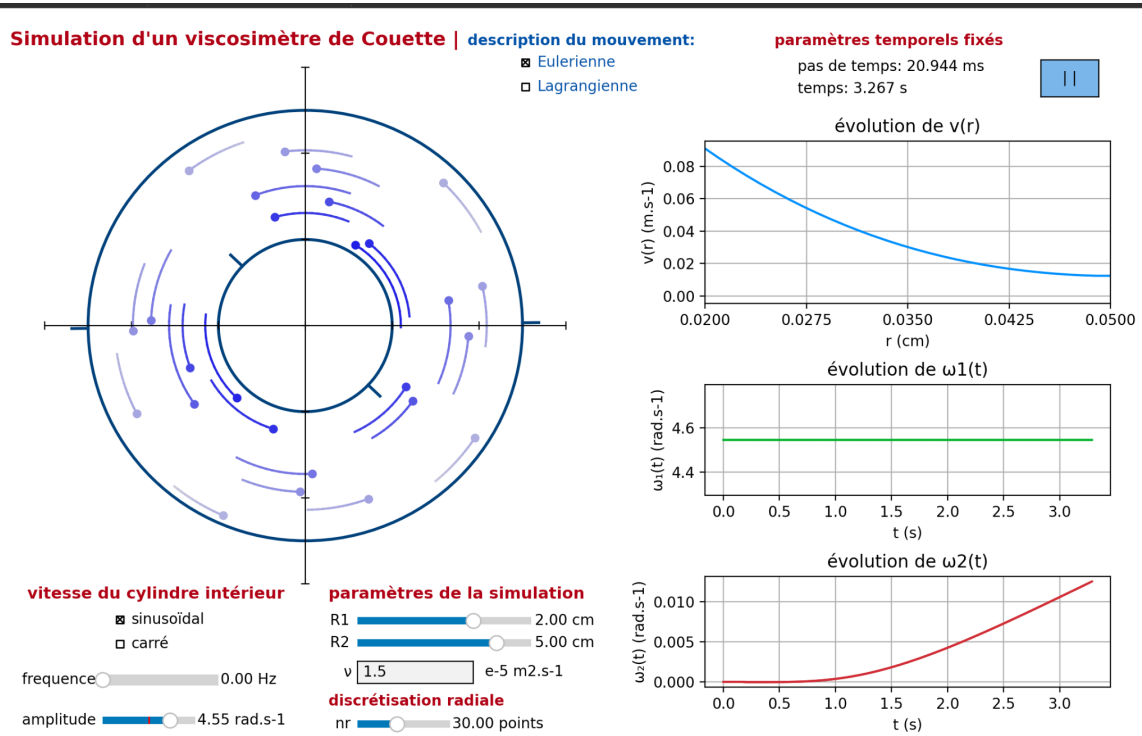


Figure 1: Final user interface of the Couette viscometer simulation. Left: animated cross-section (Eulerian mode shown). Right: live plots of $v(r)$, $\omega_1(t)$, and $\omega_2(t)$. Bottom: interactive parameter controls.

5 Results

5.1 Velocity profile convergence

The graph labelled *évolution de $v(r)$* in Figure 1 shows the tangential velocity as a function of radius at a given instant. Under constant rotation ($\omega_1 = \text{const}$), the profile converges smoothly from a linear initial state toward the well-known steady-state solution:

$$v_\theta^\infty(r) = A r + \frac{B}{r}, \quad A = -\frac{\omega_1 R_1^2}{R_2^2 - R_1^2}, \quad B = \frac{\omega_1 R_1^2 R_2^2}{R_2^2 - R_1^2},$$

which decays monotonically from R_1 to R_2 , consistent with the simulation output.

5.2 Angular velocity of the outer cylinder

The plot of $\omega_2(t)$ (Figure 1, bottom right) illustrates how viscosity progressively transfers angular momentum to the outer cylinder. Under constant inner drive, ω_2 rises monotonically and saturates. Under sinusoidal drive, ω_2 follows a smoothed, phase-lagged version of ω_1 , with the lag and amplitude governed by ν and the geometry — a physically expected behaviour.

5.3 Numerical stability

No instability or divergence was observed across the parameter ranges tested ($\nu \in [0.5, 5] \times 10^{-5} \text{ m}^2 \text{ s}^{-1}$, $N_r \in [10, 60]$, $\Delta t \approx 21 \text{ ms}$). This confirms the theoretical unconditional stability of the backward Euler scheme.

6 Conclusion

This project demonstrates that a physically meaningful fluid simulation can be built from scratch in a single week, combining rigorous numerical analysis with an interactive interface. The key design decision — choosing an implicit backward Euler scheme over an explicit one — was essential to achieving a stable, real-time simulation without imposing unrealistic constraints on the time step.

From an implementation standpoint, the use of SciPy sparse matrices and a clean object-oriented architecture made the solver both efficient and maintainable. The separation between the `Viscosimetre` model, the `BackwardsEuler` solver, and the Matplotlib display layer provides a solid foundation for future extensions, such as incorporating Crank–Nicolson time integration for second-order temporal accuracy, or adding turbulence effects for high Reynolds number regimes.

The simulation faithfully reproduces the known physics of Couette flow: smooth convergence to the analytical steady state, physically consistent angular momentum transfer, and stable behaviour under time-varying boundary conditions.

A Key Equations Summary

Governing PDE:

$$\frac{\partial v_\theta}{\partial t} = \nu \left(\frac{\partial^2 v_\theta}{\partial r^2} + \frac{1}{r} \frac{\partial v_\theta}{\partial r} - \frac{v_\theta}{r^2} \right)$$

Diffusion ratio:

$$r_{\text{diff}} = \frac{\nu \Delta t}{(\Delta r)^2}$$

Stencil (interior):

$$a_i v_{i-1}^{n+1} + d_i v_i^{n+1} + c_i v_{i+1}^{n+1} = v_i^n$$

$$a_i = -r_{\text{diff}} \left(1 - \frac{\Delta r}{2r_i} \right)$$

$$d_i = 1 + 2r_{\text{diff}} + \frac{\nu \Delta t}{r_i^2}$$

$$c_i = -r_{\text{diff}} \left(1 + \frac{\Delta r}{2r_i} \right)$$

Steady-state:

$$v_\theta^\infty(r) = Ar + \frac{B}{r}$$